

Question 1

Case Study - Cybersecurity Intrusion Detection System

A company is designing a rule-based Logical Agent to detect network intrusions. The system's knowledge base contains:

- I. If there is FailedLoginAttempts AND UnusualIPAddress -> Possible BruteForceAttack
- II. If there is FailedLoginAttempts AND OddLoginTime -> Possible AccountCompromise
- III. If there is UnusualIPAddress AND HighDataTransfer -> Possible DataExfiltration
- IV. Facts about Session X: FailedLoginAttempts = True, UnusualIPAddress = True, HighDataTransfer = True

- (a) Represent the above rules and facts precisely using Propositional Logic symbols. Define each symbol used.
- (b) Apply Forward Chaining on the knowledge base to derive all possible security alerts for Session X. Show each inference step with the rule fired and fact derived.
- (c) The goal is to confirm 'Session X indicates DataExfiltration.' Apply Backward Chaining and draw the goal reduction tree to verify this goal.

Question 2

Case Study - Airport Priority Landing System

An airport control tower uses a FOL-based knowledge base to manage priority landing clearance:

- I. for-all x: Aircraft(x) AND EmergencyDeclared(x) -> PriorityClearance(x) [Rule 1]
- II. for-all x: PriorityClearance(x) -> RunwayReserved(x) AND NOT HoldPattern(x) [Rule 2]
- III. for-all x for-all y: Aircraft(x) AND QueuedAfter(x,y) AND RunwayReserved(y) -> Delay(x) [Rule 3]
- IV. Facts: Aircraft(FlightZ), EmergencyDeclared(FlightZ), Aircraft(FlightY), QueuedAfter(FlightY, FlightZ)

- (a) Apply Unification to match Rule 1 with the given facts. Show the complete substitution set theta (MGU) at each step.
- (b) Using the Resolution method, prove that 'Delay(FlightY)' is true. Convert relevant clauses to CNF and show each resolution step.
- (c) Analyze whether the current FOL knowledge base is sufficient to handle two simultaneous emergency aircraft. Identify what additional rules or facts are needed and justify logically.

Question 3

Case Study - Industrial Equipment Maintenance Expert System

A factory maintenance logical agent operates with the following propositional knowledge base (KB):

- 1) P1: MachineOverheating -> MaintenanceNeeded
- 2) P2: MaintenanceNeeded AND HighPriorityLine -> ScheduleImmediateRepair
- 3) P3: NOT ScheduleImmediateRepair AND HighPriorityLine -> ProductionRisk
- 4) Facts: MachineOverheating = True, HighPriorityLine = True

- (a) Convert all sentences P1, P2, P3 and the facts into Conjunctive Normal Form (CNF). Show each conversion step (eliminate implication $\rightarrow$ move NOT inward $\rightarrow$ distribute).
- (b) Using Resolution Refutation, prove the theorem 'ScheduleImmediateRepair'. Negate the goal, combine with the KB in CNF, and derive the empty clause. Draw the resolution proof tree.
- (c) If the fact 'MachineOverheating' is removed from the KB, analyze step-by-step how logical conclusions change. Does 'ProductionRisk' still follow? Apply appropriate reasoning patterns to justify.

Question 4

Case Study - Minefield Robot Knowledge Agent

An AI robot navigating a minefield-clearance grid perceives: Vibration at [2,1], Smoke at [1,1], Beacon at [2,2]. The agent uses a propositional KB with these rules:

- 1) Vibration[2,1] $\rightarrow$ Mine is adjacent to cell [2,1]
- 2) Smoke[1,1] $\rightarrow$ Fire is adjacent to cell [1,1]
- 3) Beacon[x,y] $\rightarrow$ SafeZoneMarker is at [x,y]
- 4) NOT Mine[x,y] AND NOT Fire[x,y] $\rightarrow$ Safe[x,y]

- (a) Construct the agent's belief state from its percepts. Apply Modus Ponens systematically to derive new knowledge facts. List all derived conclusions.
- (b) Using Forward Chaining from the agent's percepts, identify the possible locations of the Mine and Fire. Show each reasoning step precisely.
- (c) Evaluate whether the path [1,1] $\rightarrow$ [2,1] $\rightarrow$ [2,2] is safe for the robot to reach the SafeZoneMarker. Use the knowledge derived in (a) and (b) to justify your conclusion with logical inference.

CO3 - Assessment Test 1 (Slot D)

Answer Sheet | Mithesh S | Register Number: 192565036

Topic: Logical Agents — Propositional Logic, First-Order Logic, Forward/Backward Chaining, Resolution

Question 1 — Cybersecurity Intrusion Detection System

Part (a): Propositional Logic Representation

Let the following propositional symbols be defined:

Symbol	Meaning
F	FailedLoginAttempts
U	UnusualIPAddress
O	OddLoginTime
H	HighDataTransfer
B	Possible BruteForceAttack
C	Possible AccountCompromise
D	Possible DataExfiltration

Rules expressed in propositional logic:

R1: $F \wedge U \rightarrow B$

R2: $F \wedge O \rightarrow C$

R3: $U \wedge H \rightarrow D$

Facts about Session X:

$F = \text{True}$, $U = \text{True}$, $H = \text{True}$ (O is not asserted, so it is treated as unknown/False)

Part (b): Forward Chaining

Starting from the known facts $\{F, U, H\}$, each rule is checked to see whether all of its premises are satisfied. Inference proceeds in rounds until no new fact can be derived.

Step	Rule Checked	Premises Satisfied?	Fact Derived
1	R1: $F \wedge U \rightarrow B$	$F=\text{True}, U=\text{True} \rightarrow$ satisfied	B (Possible BruteForceAttack)
2	R2: $F \wedge O \rightarrow C$	$F=\text{True}, O=\text{unknown/False} \rightarrow$ NOT satisfied	— (no new fact)
3	R3: $U \wedge H \rightarrow D$	$U=\text{True}, H=\text{True} \rightarrow$ satisfied	D (Possible DataExfiltration)
4	Re-scan all rules	No premises change; no rule newly satisfied	Fixed point reached

Final derived knowledge base: $\{F, U, H, B, D\}$. Two security alerts are raised for Session X: Possible BruteForceAttack (B) and Possible DataExfiltration (D). AccountCompromise (C) is not raised because OddLoginTime was never asserted true.

Part (c): Backward Chaining — Goal: D (Session X indicates DataExfiltration)

Backward chaining starts at the goal and searches for a rule whose conclusion matches it, then recursively proves the rule's premises as sub-goals.

Goal: D

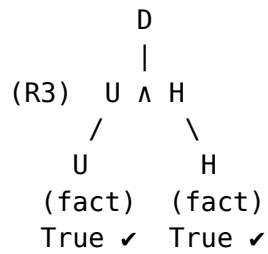
Matching rule $\rightarrow$ R3: $U \wedge H \rightarrow D$

Sub-goal 1: U ... found directly in facts ($U = \text{True}$) ✓

Sub-goal 2: H ... found directly in facts ($H = \text{True}$) ✓

Both sub-goals proven $\rightarrow$ conjunction $U \wedge H$ holds $\rightarrow$ D is PROVEN

Goal reduction tree:



Conclusion: Since both U and H are established facts, the goal 'Session X indicates DataExfiltration' is confirmed TRUE via backward chaining, consistent with the result obtained by forward chaining in Part (b).

Question 2 — Airport Priority Landing System

Given knowledge base:

Rule 1: $\forall x \text{ Aircraft}(x) \wedge \text{EmergencyDeclared}(x) \rightarrow \text{PriorityClearance}(x)$

Rule 2: $\forall x \text{ PriorityClearance}(x) \rightarrow \text{RunwayReserved}(x) \wedge \neg \text{HoldPattern}(x)$

Rule 3: $\forall x \forall y \text{ Aircraft}(x) \wedge \text{QueuedAfter}(x, y) \wedge \text{RunwayReserved}(y) \rightarrow \text{Delay}(x)$

Facts: $\text{Aircraft}(\text{FlightZ})$, $\text{EmergencyDeclared}(\text{FlightZ})$, $\text{Aircraft}(\text{FlightY})$, $\text{QueuedAfter}(\text{FlightY}, \text{FlightZ})$

Part (a): Unification with Rule 1

Rule 1's antecedent is $\text{Aircraft}(x) \wedge \text{EmergencyDeclared}(x)$. We unify each conjunct against the facts:

$\text{UNIFY}(\text{Aircraft}(x), \text{Aircraft}(\text{FlightZ})) \rightarrow \theta_1 = \{x / \text{FlightZ}\}$

Apply θ_1 to remaining conjunct: $\text{EmergencyDeclared}(x) \rightarrow \text{EmergencyDeclared}(\text{FlightZ})$

$\text{UNIFY}(\text{EmergencyDeclared}(\text{FlightZ}), \text{EmergencyDeclared}(\text{FlightZ})) \rightarrow \theta_2 = \{\}$
(identical, no new binding)

Combined MGU: $\theta = \{x / \text{FlightZ}\}$

Applying θ to the rule's conclusion gives the instantiated fact: $\text{PriorityClearance}(\text{FlightZ})$.

Part (b): Resolution Proof that $\text{Delay}(\text{FlightY})$ is True

Step 1 — Convert every sentence to CNF (clausal form):

Label	CNF Clause
C1 (Rule 1)	$\neg \text{Aircraft}(x) \vee \neg \text{EmergencyDeclared}(x) \vee \text{PriorityClearance}(x)$
C2 (Rule 2)	$\neg \text{PriorityClearance}(x) \vee \text{RunwayReserved}(x)$
C3 (Rule 3)	$\neg \text{Aircraft}(x) \vee \neg \text{QueuedAfter}(x, y) \vee \neg \text{RunwayReserved}(y) \vee \text{Delay}(x)$
C4 (fact)	$\text{Aircraft}(\text{FlightZ})$
C5 (fact)	$\text{EmergencyDeclared}(\text{FlightZ})$
C6 (fact)	$\text{Aircraft}(\text{FlightY})$
C7 (fact)	$\text{QueuedAfter}(\text{FlightY}, \text{FlightZ})$
C8 (negated goal)	$\neg \text{Delay}(\text{FlightY})$

Step 2 — Resolution derivation:

#	Resolve	Substitution	Resolvent
1	C1 with C4	$\{x / \text{FlightZ}\}$	$\neg \text{EmergencyDeclared}(\text{FlightZ}) \vee \text{PriorityClearance}(\text{FlightZ})$
2	(1) with C5	—	$\text{PriorityClearance}(\text{FlightZ})$
3	C2 with (2)	$\{x / \text{FlightZ}\}$	$\text{RunwayReserved}(\text{FlightZ})$
4	C3 with C6	$\{x / \text{FlightY}\}$	$\neg \text{QueuedAfter}(\text{FlightY}, y) \vee \neg \text{RunwayReserved}(y) \vee \text{Delay}(\text{FlightY})$
5	(4) with C7	$\{y / \text{FlightZ}\}$	$\neg \text{RunwayReserved}(\text{FlightZ}) \vee \text{Delay}(\text{FlightY})$
6	(5) with (3)	—	$\text{Delay}(\text{FlightY})$
7	(6) with C8	—	$\emptyset$ (empty clause — contradiction)

Since resolving the negated goal $\neg \text{Delay}(\text{FlightY})$ against the derived clauses yields the empty clause, the original KB together with the negated goal is unsatisfiable. Therefore, by refutation, $\text{Delay}(\text{FlightY})$ is entailed by the knowledge base. QED.

Part (c): Is the KB Sufficient for Two Simultaneous Emergency Aircraft?

No — the current KB is not sufficient. Analysis:

- Rule 1 grants $\text{PriorityClearance}(x)$ to every aircraft that independently satisfies $\text{Aircraft}(x) \wedge \text{EmergencyDeclared}(x)$. If two different aircraft, e.g. FlightZ and FlightW , both declare an emergency at the same time, Rule 1 fires for both, and Rule 2 then asserts $\text{RunwayReserved}(\text{FlightZ})$ and $\text{RunwayReserved}(\text{FlightW})$ simultaneously.

- Physically only one aircraft can occupy/reserve a given runway at a time, so the KB as written can derive a logically consistent but physically infeasible conclusion — it has no notion of mutual exclusion between runway reservations.
- There is no rule to rank or order two competing emergencies (e.g., by fuel remaining, severity of emergency, or declared time), so the system cannot decide which aircraft should land first.

Additional rules/facts needed:

- A severity or urgency predicate, e.g. $\text{Severity}(x, n)$ or $\text{FuelRemaining}(x, t)$, to compare competing emergencies.
- A precedence rule such as: $\forall x \forall y \text{ EmergencyDeclared}(x) \wedge \text{EmergencyDeclared}(y) \wedge x \neq y \wedge \text{HigherPriority}(x, y) \rightarrow \text{LandsBefore}(x, y)$.
- A mutual-exclusion constraint preventing $\text{RunwayReserved}(x)$ and $\text{RunwayReserved}(y)$ from holding simultaneously for the same runway unless $\text{SameRunway}(x, y)$ is false, e.g. $\neg(\text{RunwayReserved}(x) \wedge \text{RunwayReserved}(y) \wedge \text{SameRunway}(x, y) \wedge x \neq y)$.
- A queuing rule to place the lower-priority emergency aircraft into a holding pattern rather than simply negating HoldPattern for both.

Question 3 — Industrial Equipment Maintenance Expert System

Symbols: M = MachineOverheating, N = MaintenanceNeeded, H = HighPriorityLine, S = ScheduleImmediateRepair, R = ProductionRisk

P1: $M \rightarrow N$

P2: $N \wedge H \rightarrow S$

P3: $\neg S \wedge H \rightarrow R$

Facts: M = True, H = True

Part (a): Conversion to CNF

P1: $M \rightarrow N$

Eliminate implication: $\neg M \vee N$

(no NOT to move inward, no conjunction to distribute – already a single clause)

CNF: $\neg M \vee N$... (C1)

P2: $(N \wedge H) \rightarrow S$

Eliminate implication: $\neg(N \wedge H) \vee S$

Move NOT inward (De Morgan): $(\neg N \vee \neg H) \vee S$

Flatten disjunction: $\neg N \vee \neg H \vee S$

CNF: $\neg N \vee \neg H \vee S$... (C2)

P3: $(\neg S \wedge H) \rightarrow R$

Eliminate implication: $\neg(\neg S \wedge H) \vee R$

Move NOT inward (De Morgan): $(S \vee \neg H) \vee R$

Flatten disjunction: $S \vee \neg H \vee R$

CNF: $S \vee \neg H \vee R$... (C3)

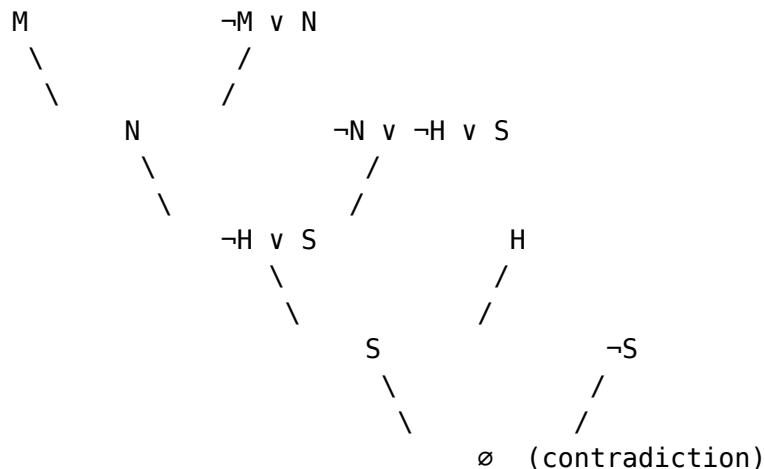
Facts as unit clauses: M ... (C4) H ... (C5)

Part (b): Resolution Refutation — Prove ScheduleImmediateRepair (S)

Negate the goal and add it to the clause set: $\neg S$... (C6)

#	Resolve	Resolvent
1	C1 ($\neg M \vee N$) with C4 (M)	N
2	C2 ($\neg N \vee \neg H \vee S$) with (1) (N)	$\neg H \vee S$
3	(2) ($\neg H \vee S$) with C5 (H)	S
4	(3) (S) with C6 ($\neg S$)	$\emptyset$ (empty clause)

Resolution proof tree:



Because the KB together with the negated goal $\neg S$ resolves to the empty clause, the KB entails S. Therefore ScheduleImmediateRepair is proven true.

Part (c): Effect of Removing the Fact 'MachineOverheating'

If M is removed from the fact base, walk the chain again:

- C1 ($\neg M \vee N$) can no longer be resolved with a unit clause M, since M is no longer asserted. N can therefore not be derived by Modus Ponens / resolution.
- Since N cannot be derived, C2 ($\neg N \vee \neg H \vee S$) cannot be resolved down to a clause containing only S — so S (ScheduleImmediateRepair) is no longer provable from the KB.
- Under the closed-world assumption normally used by rule-based expert systems (a fact not derivable is treated as false), S is now taken as False, i.e. $\neg S$ holds.
- Re-examine P3: $\neg S \wedge H \rightarrow R$. With $\neg S = \text{True}$ (by closed-world assumption) and $H = \text{True}$ (still an asserted fact), the antecedent of P3 is satisfied, so R (ProductionRisk) still fires and is derived True.

Conclusion: Yes — ProductionRisk still follows, but for a different reason than before. Previously R was blocked because S was provable ($\neg S$ was false); once MachineOverheating is removed, the repair can no longer be scheduled (S is unprovable/false), and it is precisely that absence of a scheduled repair — combined with HighPriorityLine still being true — that triggers ProductionRisk via P3. This illustrates non-monotonic behaviour: removing a fact can change downstream conclusions in P2 while still leading to the same final alert through P3.

Question 4 — Minefield Robot Knowledge Agent

Percepts: Vibration[2,1] = True, Smoke[1,1] = True, Beacon[2,2] = True

Rule 1: Vibration[2,1] $\rightarrow$ Mine is adjacent to cell [2,1]

Rule 2: Smoke[1,1] $\rightarrow$ Fire is adjacent to cell [1,1]

Rule 3: Beacon[x,y] $\rightarrow$ SafeZoneMarker is at [x,y]

Rule 4: $\neg$ Mine[x,y] $\wedge$ $\neg$ Fire[x,y] $\rightarrow$ Safe[x,y]

Part (a): Belief State via Modus Ponens

Percept (fact)	Rule Applied (Modus Ponens)	Derived Conclusion
Vibration[2,1] = True	Rule 1	A mine is adjacent to [2,1] $\rightarrow$ candidates: [1,1], [3,1], [2,2]
Smoke[1,1] = True	Rule 2	A fire is adjacent to [1,1] $\rightarrow$ candidates: [2,1], [1,2]
Beacon[2,2] = True	Rule 3	SafeZoneMarker[2,2] = True

Derived facts list: AdjacentMine([2,1]) with candidate set {[1,1],[3,1],[2,2]}; AdjacentFire([1,1]) with candidate set {[2,1],[1,2]}; SafeZoneMarker[2,2] = True.

Part (b): Forward Chaining — Locating the Mine and Fire

Step 1: Vibration[2,1]=True --(R1)--> Mine $\in$ {[1,1], [3,1], [2,2]}

Step 2: Smoke[1,1]=True --(R2)--> Fire $\in$ {[2,1], [1,2]}

Step 3: Beacon[2,2]=True --(R3)--> SafeZoneMarker[2,2]=True

Step 4 (refinement): a marked SafeZoneMarker is inconsistent with a mine at the same cell,

so [2,2] is removed from the Mine candidate set:

Mine $\in$ {[1,1], [3,1]} (narrowed from Step 1 using Step 3)

Result: the possible Mine location is [1,1] or [3,1], and the possible Fire location is [2,1] or [1,2]. Neither is pinned down to a single cell with the percepts given — more sensing (e.g. a second vibration/smoke reading from an adjacent cell) would be required to fully localize either hazard.

Part (c): Is the Path [1,1] $\rightarrow$ [2,1] $\rightarrow$ [2,2] Safe?

Rule 4 requires a positive proof of both $\neg$ Mine[x,y] and $\neg$ Fire[x,y] before a cell can be asserted Safe[x,y].

Absence of a rule concluding Mine/Fire at a cell is not the same as a proof that the cell is mine/fire-free (open-world reasoning) — so each cell on the path is checked against the candidate sets derived in Part (b):

Cell	In Mine candidates?	In Fire candidates?	Safe[x,y] provable?
[1,1]	Yes (from Step 4)	No	NOT provable — cannot confirm $\neg$ Mine[1,1]
[2,1]	No	Yes (from Step 2)	NOT provable — cannot confirm $\neg$ Fire[2,1]
[2,2]	No (excluded, Step 4)	No	Provable — SafeZoneMarker also supports this

Conclusion: The path [1,1] $\rightarrow$ [2,1] $\rightarrow$ [2,2] is NOT confirmed safe. Both [1,1] and [2,1] remain in the unresolved hazard-candidate sets derived in Part (b), so Rule 4's premises ($\neg$ Mine $\wedge$ $\neg$ Fire) cannot be proven for either cell — the robot cannot logically justify traversing them. Only [2,2] satisfies Rule 4, supported additionally by its SafeZoneMarker from Rule 3. The agent should not take this route without further percepts to eliminate [1,1] and [2,1] as hazard locations, or should route around them entirely.

